

Personalized Hotel Search Ranking on the Expedia 2013 Dataset

Hendrico Burger¹, Hidde Franke¹, and Joanne Pals¹

Vrije Universiteit Amsterdam, Faculty of Science, Amsterdam, the Netherlands
{h.burger,h.franke,j.pals}@student.vu.nl

1 Introduction

This report describes the entry of team VU-DM-2026-Group-5 in the Data Mining Techniques 2026 ranking competition on the Expedia 2013 personalized search dataset. The task is to rank, for each search `srch_id`, the candidate hotel properties so that booked properties surface above clicked-only and non-interacted ones. The metric is NDCG@5 with relevance grades 5 (booked), 1 (clicked), and 0 (no interaction). The competition is run on Kaggle; the public leaderboard is computed on a fraction of the test set and is used here only as an external sanity check, never as a tuning target.

Our final selected submission is a three-seed LightGBM LambdaMART model on 49 raw features plus 12 leakage-safe historical priors, score-averaged across seeds. It reaches $\text{NDCG@5} = 0.39690$ on the public Kaggle leaderboard (verified upload), against a strong heuristic baseline of 0.2836 (sort by `prop_location_score2` descending) on our locked holdout, and against a safe LightGBM baseline of 0.39500 on the public leaderboard.

The report emphasizes three threads. First, a clean separation between features and modelling choices that survived ablation and those that did not, with same-seed deltas as the acceptance signal. Second, explicit leakage controls in target encoding via 5-fold group K-fold on `srch_id`. Third, an honest accounting of the local-to-public shrinkage observed across the successive model variants, including the deliberate decision not to upload a holdout-maximising rank blend.

2 Related work and primary references

We position the work against three established threads. **LambdaMART** (the boosting view of the LambdaRank objective) is the workhorse of pairwise and listwise learning to rank; we use its LightGBM implementation **LGBMRanker**. **Target encoding for high-cardinality categoricals** provides a general framework for the historical priors we add on top of the anchor; we use Laplace smoothing combined with K-fold group splitting to keep the encoding leakage-safe. **Fairness in ranking** motivates our exposure-disparity mitigation: we measure exposure@5 by segment and apply a post-processing additive rerank with a single tunable hyperparameter λ . The most directly comparable prior work is the VU 2021 student report by Bellucci, Hu, and Lin [1], who train a LightGBM

LambdaMART model on the same in-class Kaggle leaderboard with NDCG@5 as the metric and report 0.41726 public NDCG@5; we use that report as a sanity-check upper bound, not as a target, and we explicitly avoid the parts of their pipeline (notably leave-one-out per-property encoding without K-fold) that we judge less leakage-safe than the encoding scheme of Section 4.2.

3 Data and metric

3.1 Dataset

The training set contains 4,958,347 rows over 54 columns; the test set contains 4,959,183 rows over 50 columns, missing four train-only columns: `position`, `click_bool`, `booking_bool`, and `gross_bookings_usd`. Both slices span 2012-11-01 to 2013-06-30. Click rate on train is 4.475%, booking rate 2.791%; on average each `srch_id` contains roughly one booking. Search lists range from 5 to 38 properties (mean 24.82, median 29) with a bimodal distribution that peaks between 30 and 33; see Figure 1. The median list length is well above 5, so NDCG@5 captures the part of the list a user typically scans.

Heuristic single-feature baselines on the locked holdout: random ranking averaged over 3 seeds 0.1579 (std 0.0008), sort by `prop_starrating` descending 0.1949, sort by `price_usd` ascending 0.1987, sort by `prop_location_score2` descending 0.2836. The last baseline is the strongest single-feature heuristic and we use it as the floor a learned model has to clear. Figure 2 shows that no individual numeric feature exceeds an absolute Spearman correlation with `booking_bool` of 0.10 on a five percent search-stratified sample, which is consistent with the heuristics landing at most around 0.28.

3.2 Metric

For a search q with predicted ordering $p_1, p_2, \dots$ and relevance grades $r(p_i) \in \{0, 1, 5\}$ we compute

$$\text{DCG@5}(q) = \sum_{i=1}^5 \frac{r(p_i)}{\log_2(i+1)}, \quad \text{NDCG@5}(q) = \frac{\text{DCG@5}(q)}{\text{IDCG@5}(q)}, \quad (1)$$

with $\text{IDCG@5}(q)$ defined as the DCG@5 of the descending sort of r . By convention we set $\text{NDCG@5}(q) = 0$ when $\text{IDCG@5}(q) = 0$; on this data the share of such queries on train and the locked holdout is 0%, so the convention does not move any reported number. The model’s labels use Variant A label encoding: relevance $\{0, 1, 5\}$ is remapped to label index $\{0, 1, 2\}$ with `label_gain` = [0, 1, 5], so the LambdaRank objective optimizes the assignment-graded NDCG directly.

3.3 Splits

The locked holdout (sha256 prefix `bc8ea6f6`) contains 19,980 `srch_id` queries, group-disjoint from the training and validation slices (159,835 and 19,980 queries

respectively). The same JSON file is used across all seeds and all iterations; only the train and validation rotation changes per seed. This makes results from different iterations directly comparable on the same set of held-out searches.

4 Method

4.1 Anchor features

The anchor consists of 49 features: 19 raw numeric columns covering visitor history, property quality, price, search context, and the `random.bool` flag; 24 raw competitor columns (`comp1..8.rate`, `comp1..8.inv`, `comp1..8.rate.percent.diff`); five categorical identifiers (`site_id`, `visitor_location_country_id`, `prop_country_id`, `prop_id`, `srch_destination_id`); and one derived flag `has_visitor_history` that is 1 when both `visitor_hist_starrating` and `visitor_hist_adr_usd` are non-null. The five identifiers are passed to LightGBM via `categorical_feature`; this matters because `prop_id` can absorb per-property biases directly inside the trees, leaving less room for explicit per-property aggregates.

4.2 Leakage-safe historical priors

On top of the 49 anchor features we add 12 historical priors from three blocks: keyed by `prop_id`, by `srch_destination_id`, and by the pair (`prop_id`, `srch_destination_id`). Each block contributes four features: a log impressions count `*_impressions.log1p`, a smoothed click rate `*_click.rate.smooth`, a smoothed booking rate `*_booking.rate.smooth`, and a smoothed mean relevance `*_relevance.mean.smooth`. Smoothing is Laplace style:

$$\hat{r}_g = \frac{\sum_{i \in g} y_i + \alpha \bar{y}_{\text{global}}}{|g| + \alpha}, \quad \alpha = 20. \quad (2)$$

For training rows we use 5-fold group K-fold on `srch_id` with `fold_seed=42`: a row in fold f receives priors aggregated over the other 4 folds, never from rows in the same query nor from itself. For validation, holdout, and test rows we use full-train aggregates, which is leakage-safe because those queries are group-disjoint from train. We never aggregate over the union of train and test, even when prior work has done so. Unseen groups fall back to the global rate by construction: when $|g| = 0$, Equation 2 reduces to $\bar{y}_{\text{global}}$.

4.3 Model

The primary model is LightGBM `LGBMRanker` with the `lambdarank` objective. Hyperparameters: `depth 9`, `num_leaves = 28`, learning rate 0.05, `bagging_fraction = 0.958`, `feature_fraction = 0.927`, `min_child_samples = 50`, `label_gain = [0, 1, 5]`. Threading is pinned (`num_threads=6`); we never use `-1`. During the split phase we fit with up to 1000 trees and early stop with patience 50 on the validation slice; for full-train deployment we use the fixed best-iteration values learned during the split phase (seed 42: 291, seed 123: 279, seed 456: 375). The final

submission is the score-average over the three seeds. As a calibration baseline we ran XGBoost `rank:ndcg` with comparable hyperparameters on a five percent search-stratified sample; this is reported in Section 5.3.

5 Experiments

5.1 Hypothesis-driven ablations that did not survive

We tested five hypothesis blocks on the locked holdout, each evaluated by same-seed delta against the anchor over three seeds (42, 123, 456); the acceptance threshold was $+0.0011$ NDCG@5, set above the anchor seed-to-seed standard deviation. None passed.

- Per-search z-score and rank features on price, star, and location: mean delta -0.000327 , reject.
- Per-`prop_id` leave-one-out click-rate and conversion-rate priors: mean delta -0.000525 , reject.
- Mean-position proxy from `random.bool=0` train rows used as a test-time proxy: mean delta $+0.000545$, sub-noise, reject.
- Negative downsampling at 1:1 and 1:4: mean deltas -0.016920 and -0.005600 , reject.
- Competitor aggregates and Liu-style composite features: deltas near zero on a single seed pilot, reject.

The most likely explanation for the per-`prop_id` leave-one-out priors and the mean-position proxy is that the anchor’s categorical `prop_id` already carries the property-level information those features sought to add explicitly; the smoothed priors of Section 4.2 succeed where these explicit features fail because they are aggregated over coarser keys (and over pairs of keys) that the trees cannot reach via a single `prop_id` split.

5.2 Final ablation table

Table 1 traces the path from the strongest heuristic baseline to the final selected model. Public scores are reported only for verified Kaggle uploads; local-only configurations are marked accordingly. The reported gaps are local holdout NDCG@5 minus public NDCG@5; we revisit the trend in Section 7.3.

5.3 Second technique: XGBoost calibration

We compared LightGBM against XGBoost `rank:ndcg` on a five percent stratified-by-`srch_id` sample of the training set with identical features. On the sample LightGBM scored $\text{NDCG@5} = 0.329601$ and XGBoost $\text{NDCG@5} = 0.356409$. Full-data XGBoost did not finish within our compute budget on a single laptop, so XGBoost serves as a calibration baseline rather than a production model. The two implementations agree on the direction of the top feature importances, which gives us additional confidence in the LightGBM ranking.

Table 1. Ablation across the model variants explored. Local NDCG@5 is on the locked holdout (sha256 bc8ea6f6); public NDCG@5 is the Kaggle public leaderboard score. *Local only* means the candidate was generated and validated locally but never uploaded.

configuration	local NDCG@5	public NDCG@5 status
Location-score heuristic	0.2836	— local floor
LightGBM LambdaMART anchor, individual seeds	0.392667	— individual mean ($\sigma = 0.001070$)
3-seed anchor score average, submitted safe baseline	0.395289	0.39500 uploaded as the safe baseline
5-seed anchor ensemble	0.397348	— local only
5-seed anchor ensemble with post-hoc blend	0.398246	— local only
Diverse LightGBM ensemble with post-hoc blend	0.399638	0.39685 uploaded
Historical-prior 3-seed ensemble, final submission	0.400682	0.39690 final selected
Conservative rank aggregation, no upload	0.400453	— not selected, no upload
Holdout-max rank aggregation variant, not selected	0.401482	— holdout max, not selected
Additional visitor-country prior block, rejected	+0.000745 delta	— rejected, below threshold

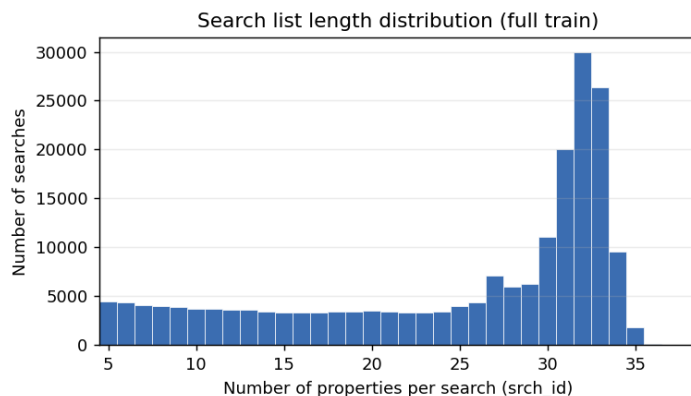


Fig. 1. Search-list length distribution on the full training set. Bimodal: a flat low-frequency segment from length 5 to 25 and a sharp peak at 30 to 33. Median 29.

6 Results

Figures 1, 2, 3, and 4 are referenced from the surrounding discussion. The feature importance figure shows that `prop_id` has the highest total gain by a margin, that `prop_location_score2` and `srch_destination_id` are essentially tied for second and third place, and that the strongest prior feature (`prop_dest_click_rate_smooth`) ranks sixth, ahead of `prop_log_historical_price` and the rest of the anchor.

7 Bias, deployment, and limitations

7.1 Bias detection

We segment the holdout along two dimensions and report NDCG@5 plus exposure@5, which we define as the share of in-search positives that surface in the top 5. By visitor country, the largest segments land between 0.350 and 0.416

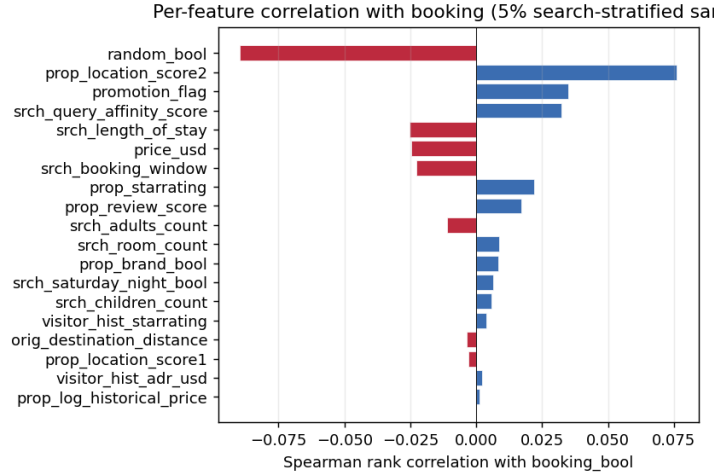


Fig. 2. Spearman rank correlations of numeric features with `booking_bool` on a five percent search-stratified sample. The strongest absolute correlations are `random_bool` (-0.0896) and `prop_location_score2` ($+0.0760$); all other features sit below 0.04 . No single feature can rank well on its own.

NDCG@5 with exposure close to 1.0; the lowest score is for country 99 (0.349878 NDCG@5, exposure 0.9612). By property popularity tertile (computed from total training impressions per `prop_id`) the picture is sharper: the high tertile reaches exposure 0.9266 with NDCG@5 0.390424, the medium tertile 0.2651 / 0.395074, and the low tertile 0.2010 / 0.426924. The low tertile has the highest per-search NDCG when those hotels appear; they simply do not appear in the top 5 often. We therefore frame this as exposure disparity, not as a discrimination claim: high-popularity properties dominate the top of the ranking, and that visibility gap propagates to future training data.

7.2 Mitigation: post-processing exposure rerank

We mitigate with a single post-processing step. For each property p in popularity tertile $t(p)$ with prior probability $P_{t(p)}$ on train, we rerank by

$$\tilde{s}(p) = s(p) + \lambda \cdot (-\log P_{t(p)}), \quad \lambda = 0.20, \quad (3)$$

which leaves the original ranker score $s(p)$ untouched but adds a tertile-prior penalty that grows for under-represented tertiles. On the locked holdout, $\lambda = 0.20$ produces the trade-off in Table 2: a global NDCG@5 drop of 0.003090 buys a 5.3 percentage point gain in medium-tertile exposure and a 2.2 percentage point gain in low-tertile exposure. The high tertile loses 3.0 percentage points. The primary Kaggle submission is unmodified; the rerank is a deployment-time variant.

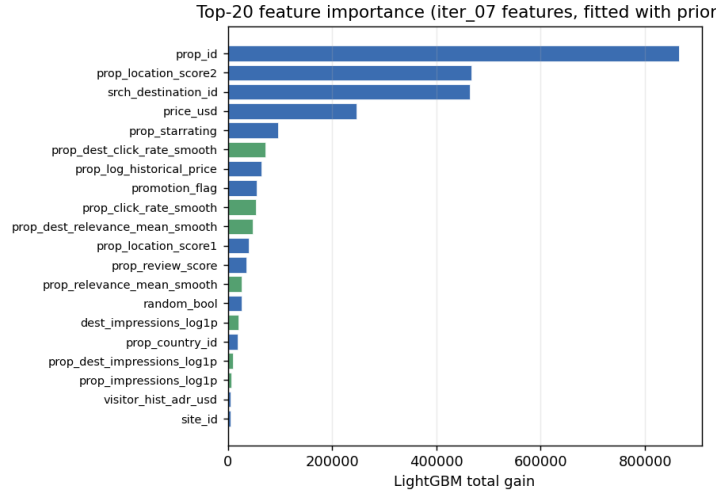


Fig. 3. Top-20 features by LightGBM total gain in a model fitted on the 49 anchor features plus the 12 leakage-safe priors. Categorical `prop_id` dominates, followed by `prop_location_score2`, `srch_destination_id`, `price_usd`, and `prop_starrating`; the strongest prior feature `prop_dest_click_rate_smooth` ranks sixth.

Table 2. Popularity tertile exposure and NDCG@5 before and after the post-processing rerank with $\lambda = 0.20$.

tertile	exposure before	exposure after	NDCG@5 before	NDCG@5 after
high	0.9266	0.8965	0.390424	0.381779
medium	0.2651	0.3184	0.395074	0.422510
low	0.2010	0.2227	0.426924	0.442720

7.3 Honest local-to-public discussion

Across the verified Kaggle uploads the local-to-public gap moved in one direction: Table 3 shows the trajectory.

The direction of improvement transferred (public moved from 0.39500 to 0.39685 to 0.39690), but the magnitude shrank: the local-to-public gap of +0.000289 on the safe baseline became -0.003782 on the final model. Each individual decision was leakage-safe by construction (Group K-fold on `srch_id`, no train-test joint statistics, locked holdout, fixed `prop_id` test set), but the cumulative exploratory budget on the same locked holdout introduced a second-order optimism: we kept whichever change scored highest on that fixed set, so the expected public improvement was overstated. The reported public 0.39690 should be read as a verified outcome and not as a prediction of the local 0.400682 transferring.

Anti-overfitting decision in the final exploratory stage: we identified a rank-blend candidate (a holdout-max rank aggregation variant, weighted 0.60 final +

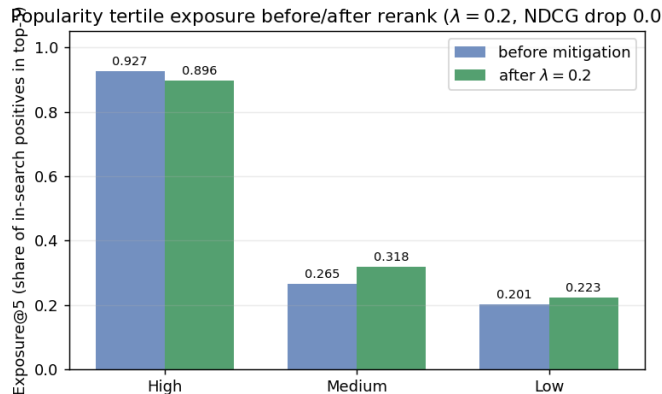


Fig. 4. Popularity tertile exposure@5 before and after the post-processing rerank with $\lambda = 0.20$ on the locked holdout. The high tertile loses about 3 percentage points of exposure; the medium and low tertiles gain about 5 and 2 percentage points respectively. The corresponding global NDCG@5 drop is 0.003090.

Table 3. Local holdout NDCG@5 versus Kaggle public NDCG@5 for the three uploaded submissions. Local holdout is on the locked split; public is the Kaggle public leaderboard.

submission	local	public	gap
Submitted safe baseline	0.395289	0.39500	+0.000289
Diverse ensemble submission	0.399638	0.39685	−0.002788
Historical-prior final submission	0.400682	0.39690	−0.003782

0.30 diverse + 0.10 safe) that scored 0.401482 on the locked holdout, +0.000800 above the final selected model. We did not select it. Selecting the holdout-max of a small grid is exactly the move the shrinkage trajectory warns against; we instead followed a pre-specified heuristic (simplest non-trivial blend with at least two methods and holdout ≥ 0.398) which picked the conservative weighting at 0.400453, and we did not upload that either. The headline submission remains the simpler historical- prior 3-seed model.

7.4 Deployment notes

We sketch a deployment pattern faithful to what the codebase already supports.

Request-time scoring. For an incoming search the model receives the candidate property list together with the search-context fields. Anchor features and the 12 priors are computed per row. The K-fold encoding is training-time only: at request time we use the full-train aggregates (Section 4.2), which is leakage-safe

because incoming searches are group-disjoint from training searches by construction. Scoring is batched per search (groups of 5 to 38 properties), which fits comfortably in a single LightGBM `predict` call.

Cold-start hotels. A new `prop_id`, a new `srch_destination_id`, or a new pair has zero impressions in train. The Laplace smoothing in Equation 2 reduces to $\bar{y}_{\text{global}}$ for those keys, so the model still receives a sensible prior signal and falls back to the raw anchor features (`price_usd`, `prop_starrating`, `prop_location_score1/2`, `promotion_flag`). No separate cold-start path is needed.

Monitoring. We recommend tracking three families of metrics. (i) Offline NDCG@5 monthly on a held-out slice with the same group-aware protocol. (ii) Click and book rate on the served top-5 weekly, since these are model-free proxies that catch obvious regressions. (iii) Exposure@5 by popularity tertile and by visitor country, weekly, with an alert if the high-tertile share rises above the pre-mitigation level. The fairness-aware variant ($\lambda = 0.20$, Section 7.2) is an opt-in deployment switch with a known NDCG@5 cost of 0.003090 on the locked hold-out.

Retraining cadence. Anchor features and the LightGBM model are retrained monthly on the rolling training window. The 12 priors are refreshed on the same cadence and recomputed per fold from the new training window; we do not stream the priors at request time.

Limitations. Three limitations carry into deployment. First, the local-to-public shrinkage in Section 7.3 suggests holdout-based selection loses calibration after many iterations; we recommend a single end-to-end holdout run or nested cross-validation for any future evaluation pass. Second, we did not validate the model on a stricter time-based split, which would be the right protocol if test were genuinely in the future; here train and test cover the same date range so group-aware splitting is sufficient. Third, the bias evaluation uses popularity tertiles and visitor country with adequate-support thresholds (at least 500 searches and 100 positives per segment); finer or intersectional segments are not reported.

8 Conclusion

A LightGBM LambdaMART model on 49 anchor features plus 12 leakage-safe historical priors reaches public NDCG@5 = 0.39690 on this year’s instance of the assignment. The path from a heuristic floor at 0.2836 (locked holdout) to a verified public score of 0.39690 was driven by a small set of carefully ablated choices (anchor feature design, label-gain calibrated to the assignment relevance grades, score-averaged 3-seed ensembling, K-fold group-safe historical priors). The path from the safe baseline at 0.39500 public to 0.39690 public was much smaller, and the local gain shrank in transfer; we report the gap honestly. Two extension directions that did not pay off (a holdout-max rank blend and a fourth prior

block keyed by visitor country) are documented in the ablation table as negative results. The deployment design supports a fairness-aware exposure rerank as an opt-in switch with a measured NDCG@5 cost.

References

1. Bellucci, T., Hu, Q., Lin, C.C.: On the ranking of Expedia search results using LambdaMART. Group 55, Data Mining Techniques 2021, Vrije Universiteit Amsterdam (2021)